

Department of Physical Science
Faculty of Applied Science, University of Vavuniya
IT2143 (P) - Visual Programming (P)
In-Course Assessment - 01

Time Allowed: 50 Minutes

Date: 25.10.2024

Create a folder in desktop with your “Registration Number” (e.g:2021/ICT/01) and save the answer (C# console application project) for the following question.

Question 01

Write a C# program to manage the inventory of a small bookstore. The bookstore sells different categories of books: Fiction, Non-fiction, and Children's books. The program should meet the following requirements:

1. Create a Book class with properties:
 - Title (string)
 - Author (string)
 - Category (string) (Fiction/Non-fiction/Children)
 - Price (double)
 - *Constructor*: Initializes a book with its title, author, category, and price.
 - *Method*: DisplayBookDetails() to displays the details of the book.
2. In the main program, implement a menu that allows the user to perform the following actions:
 - i. **Add a new book:**
 - a. The user is asked to enter the book details.
 - b. Category Selection:
 - The user selects a category from the given options (Fiction, Non-fiction, Children's books).
 - c. Price Input Validation:
 - The program uses a try-catch block to ensure the price entered is a valid number. If the price is zero or negative, an error message is shown, and the book is not added to the inventory.
 - d. Once a valid book is entered, it is stored in the inventory array, and the program confirms that the book was added successfully.
 - ii. **Display all books:** List all books currently stored in the inventory with their details.
 - iii. **Search for books by category:**
 - a. The user is prompted to enter a category (e.g., Fiction, Non-fiction, Children's).
 - b. The program searches through the inventory and displays all the books that match the entered category.
 - c. If no books match the category, a message is displayed informing the user that no books were found.
 - iv. **Calculate and display the total value of books:** Compute the sum of the Price property of all books and display the total value of the inventory.
 - v. **Exit the program:** Terminate the application when the user selects this option.

- **Error Handling in Menu**

If the user selects an invalid option (anything other than 1 to 5), the program shows an error message and asks the user to select a valid option. This helps prevent the program from crashing due to incorrect input.

- **General Flow**

After the user performs an action (adding a book, searching, etc.), the program always returns to the main menu, allowing the user to perform another action or exit.

The program will keep running until the user explicitly chooses the "Exit" option.

Sample Data

Book Title	Author	Category	Price (USD)
The Great Gatsby	F. Scott Fitzgerald	Fiction	15.99
Sapiens: A Brief History of Humankind	Yuval Noah Harari	Non-fiction	20.50
Harry Potter and the Sorcerer's Stone	J.K. Rowling	Children's	12.99
1984	George Orwell	Fiction	10.75
Becoming	Michelle Obama	Non-fiction	18.00

Sample outputs

```
Bookstore Inventory Menu:
1. Add a book
2. Display all books
3. Search books by category
4. Calculate total value of books
5. Exit
Choose an option: 1
Enter book title: 1984
Enter author name: George Orwell
Select Category:
1. Fiction
2. Non-fiction
3. Children's
Choose category: 1
Enter book price: 0
Error: Price must be greater than zero.

Bookstore Inventory Menu:
1. Add a book
2. Display all books
3. Search books by category
4. Calculate total value of books
5. Exit
Choose an option: |
```

```
Bookstore Inventory Menu:
1. Add a book
2. Display all books
3. Search books by category
4. Calculate total value of books
5. Exit
Choose an option: 1
Enter book title: The Great Gatsby
Enter author name: F. Scott Fitzgerald
Select Category:
1. Fiction
2. Non-fiction
3. Children's
Choose category: 1
Enter book price: 15.99
Book added successfully!
```

Question 02

You are required to develop a simple Employee Management System in C#. The system should manage employee records and perform various operations. The requirements are as follows:

Step 1: Create the Class Library Project

Create a new Class Library project named **EmployeeManagement** that contains the Enums, Employee partial classes and the delegate.

1. **Enums:** Define an enum called **EmployeeRole** that represents different roles (Manager, Developer, Designer, Tester) an employee can have.

2. Partial Classes: Create a partial class named **Employee**.
 - 2.1. In one part of the class, define the properties of the employee:
 - Name (string)
 - EmployeeId (int)
 - Role (EmployeeRole)
 - Salary (double)
 - 2.2. In the other part of the class, define a method called `DisplayEmployeeDetails()` that prints the employee details to the console.
3. **Delegates:** Define a delegate called **SalaryUpdated** that takes an *Employee* object as a parameter. This delegate will be used to notify when an employee's salary is updated.

Step 2: Create the Main Application Project

Create a new Console Application project named **EmployeeManagementApp**. In this project, reference the **EmployeeManagement** DLL.

- In the Main method of your application:
- An instance of the Employee class is created with initial values.
 - The initial employee details are displayed using the **DisplayEmployeeDetails** method.
 - The salary is updated, and the **SalaryUpdatedHandler** delegate is invoked to notify of the salary change.
 - Finally, the updated employee details are displayed.

Sample output

```
Name: Alice Smith, Employee ID: 101, Role: Developer, Salary: $60,000.00
The salary of Alice Smith has been updated to 65000.
Name: Alice Smith, Employee ID: 101, Role: Developer, Salary: $65,000.00
|
```